# Tailwind CSS 从 0 到 1：构建可维护的 Utility-First 前端样式体系

**以 Tailwind CSS v4.3 官方文档为主线 · React + Vite 完整实践**

---

## 本教程目录


1. 第 1 章：先建立正确的心智模型
2. 第 2 章：从空项目安装 Tailwind CSS v4
3. 第 3 章：读懂 utility 类名并构建视觉基础
4. 第 4 章：真正理解 Flex、Grid、溢出与定位
5. 第 5 章：响应式设计——从移动端开始，而不是最后补救
6. 第 6 章：交互状态——hover、focus、active、disabled、group 与 peer
7. 第 7 章：深色模式、主题令牌与语义颜色
8. 第 8 章：可维护的 className、组件 API 与样式职责
9. 第 9 章：类名扫描机制、动态类名陷阱与 safelist
10. 第 10 章：Tailwind v4 与 v3 的配置差异
11. 第 11 章：基础 CSS、层叠模型与可访问性仍然重要
12. 第 12 章：完整实践——做一个响应式学习仪表盘
13. 第 13 章：常见故障排查
14. 第 14 章：学习完成后的工程化自检清单
15. 第 15 章：下一步如何继续学习
16. 官方资料索引


## 如何学习这份教程

请按章节动手，不要试图先记住所有类名。每一章都围绕一个实际问题展开：先理解浏览器最终要完成什么，再学习 Tailwind 如何表达它，最后把它放回一个完整页面里验证。建议每完成一章，就在自己的项目中复现代码，并刻意修改一个值，观察页面会发生什么。

本教程以 Tailwind CSS v4 的官方文档为准。官方资料会持续更新，所以开始新项目或升级项目时，应先确认项目锁定的主版本，再阅读相同主版本的文档：

- 官方文档首页：https://tailwindcss.com/docs
- Vite 安装：https://tailwindcss.com/docs/installation/using-vite
- Utility-first 样式思想：https://tailwindcss.com/docs/styling-with-utility-classes
- 响应式设计：https://tailwindcss.com/docs/responsive-design

- 状态变体：https://tailwindcss.com/docs/hover-focus-and-other-states
- 深色模式：https://tailwindcss.com/docs/dark-mode
- 主题变量：https://tailwindcss.com/docs/theme
- 类名扫描与 safelist：https://tailwindcss.com/docs/detecting-classes-in-source-files
- 自定义样式：https://tailwindcss.com/docs/adding-custom-styles
- v3 升级到 v4：https://tailwindcss.com/docs/upgrade-guide
- 浏览器兼容性：https://tailwindcss.com/docs/compatibility

# 第 1 章：先建立正确的心智模型

## 1.1 Tailwind CSS 是什么

浏览器最终只认识 CSS。无论你使用传统 CSS、CSS Modules、Sass，还是 Tailwind CSS，最后都需要产生类似 `display: flex`、`padding: 1rem`、`background-color: ...` 的 CSS 声明。

Tailwind CSS 是一个 utility-first CSS 框架。它提供很多职责单一的工具类（utility classes），每个类通常对应一个明确的样式意图。例如：

```
<div class="flex gap-4 rounded-xl bg-white p-6 shadow-sm">
  <h2 class="text-xl font-semibold text-slate-900">课程标题</h2>
</div>
```

这些类并不是神秘语法，而是对常见 CSS 属性的简短表达：

```
/* 用于理解的等价 CSS，真实生成结果会由 Tailwind 管理 */
.card-like-element {
  display: flex;
  gap: 1rem;
  border-radius: 0.75rem;
  background-color: white;
  padding: 1.5rem;
  box-shadow: 0 1px 2px 0 rgb(0 0 0 / 0.05);
}

.card-like-element h2 {
  font-size: 1.25rem;
  font-weight: 600;
  color: #0f172a;
}
```

两种方式都能工作。不同点在于，传统 CSS 倾向于先定义有语义的选择器，再在 CSS 文件中写声明；utility-first 则把许多小而清楚的样式组合放在元素附近。

## 1.2 为什么叫 utility-first

“Utility-first”不是“每个元素都随便堆类名”，而是先用低耦合、单一职责的样式工具构造界面，再在确实存在重复模式时抽象组件或设计令牌。

比如一个按钮需要：

- 作为行内弹性容器摆放图标和文字：`inline-flex items-center gap-2`
- 有可点击的内边距：`px-4 py-2`
- 有圆角和背景：`rounded-lg bg-indigo-600`
- 有清晰文字：`text-sm font-semibold text-white`
- 悬停时改变背景：`hover:bg-indigo-700`
- 键盘操作时显示焦点轮廓：`focus-visible:outline-2 focus-visible:outline-offset-2`

组合起来：

```
<button
  class="inline-flex items-center gap-2 rounded-lg bg-indigo-600 px-4 py-2 text-sm font-semibold text-white hover:bg-
indigo-700 focus-visible:outline-2 focus-visible:outline-offset-2"
>
  保存更改
</button>
```

关键思路是：类名描述“这个元素应该呈现成什么样”，而不是一定先为它取一个抽象的 CSS 类名。

## 1.3 这不是“不要 CSS”

不要把 Tailwind 理解成 CSS 的替代品。你仍然需要理解盒模型、继承、层叠、选择器、定位、可访问性、语义 HTML，以及哪些 CSS 规则会影响布局。Tailwind 帮你更快地表达常见样式，也帮助团队约束设计尺度，但不会替你决定页面语义是否正确。

常见误区：

1. **只会抄类名，不理解属性。** 当布局溢出时，你不知道应该检查 `min-width`、`overflow` 还是 flex 子项的收缩行为。
2. **所有样式都塞进超长的 `className`。** 组件的职责和设计令牌没有边界，重复片段越来越难维护。
3. **把每个状态都用颜色表示。** 没有键盘焦点、禁用状态或语义化标签，视觉上看似完成，真实使用却很困难。
4. **动态拼接类名。** 浏览器运行时看起来有字符串，但构建工具扫描不到完整类名，最终 CSS 没有生成。

本教程会逐步避开这些陷阱。

## 1.4 Tailwind 如何生成 CSS

Tailwind 在构建时扫描 HTML、JSX、TSX 等项目文件，寻找可能的类名，然后为识别到的类生成所需 CSS。它不需要在浏览器运行时为每个组件动态计算整套样式，因此常规生产使用不会因为每个 `className` 而引入运行时样式引擎。

这一机制也带来一个重要规则：**类名要以构建工具能在源码中识别的完整字符串出现。** 第 9 章会专门讲解。

### 本章练习

1. 写一个 `div`，使用 `flex p-4 gap-3 rounded-xl shadow-sm`。
2. 逐一说出每个类名会影响什么 CSS 属性。
3. 暂时去掉 `p-4` 或 `gap-3`，观察元素间距差异。

如果你能把 utility 类名翻译成 CSS 的布局或外观意图，就进入下一章。

---

# 第 2 章：从空项目安装 Tailwind CSS v4

本教程使用 React + Vite 作为演示环境。Tailwind 同样可用于普通 HTML、Vue、Svelte 和其他构建环境，但安装步骤会不同；不要把某个框架的安装方法直接复制到另一个框架。

## 2.1 先检查环境

需要安装 Node.js。建议使用仍在维护的 LTS 版本。安装后打开终端，运行：

```
node -v
npm -v
```

两条命令都应输出版本号。如果系统提示找不到命令，先解决 Node.js 和 PATH 配置，不要急着安装 Tailwind。

## 2.2 创建 React + Vite 项目

在终端执行：

```
npm create vite@latest tailwind-learning -- --template react
cd tailwind-learning
npm install
```

如果命令询问是否继续，按提示操作。创建完成后先运行原始项目：

```
npm run dev
```

终端会显示本地开发地址，一般是 `http://localhost:5173`。在浏览器打开它，确认默认 Vite 页面能够正常显示，然后停止开发服务器（终端按 `Ctrl+C`）。先验证基础项目很重要：如果安装 Tailwind 后出现问题，你就知道问题是新增配置导致的，而不是原项目本来就不能运行。

## 2.3 安装 v4 的 Vite 插件

按照当前官方 Vite 安装方式，安装 Tailwind 与专用 Vite 插件：

```
npm install tailwindcss @tailwindcss/vite
```

对于正式项目，务必提交 `package-lock.json`，通过锁文件保证团队使用可重复的依赖版本。不要今天安装 v4，过几个月又无意中把项目升级到新主版本。主版本升级要作为一次明确的工程变更来处理。

## 2.4 配置 `vite.config.js`

打开项目根目录的 `vite.config.js`，将其调整为：

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

这里有两个插件：`react()` 负责 React 相关的 Vite 支持，`tailwindcss()` 让 Tailwind 参与构建流程。不要把 v3 的 `tailwindcss` PostCSS 插件写法直接照搬到 v4 的 Vite 项目中。

## 2.5 建立 CSS 入口

打开 src/index.css，先清掉 Vite 默认演示样式，改为：

```
@import "tailwindcss";
```

然后确认 src/main.jsx 引入了这个 CSS 文件：

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

如果 main.jsx 中还引入了 ./App.css 或 ./style.css，并且这些文件里有影响布局的默认样式，请暂时移除相关导入或清理内容，避免样式冲突。

## 2.6 验证安装是否成功

把 src/App.jsx 临时替换为：

```
export default function App() {
  return (
    <main className="min-h-screen bg-slate-50 p-8">
      <h1 className="text-3xl font-bold text-indigo-700">
        Tailwind 已经开始工作
      </h1>
      <p className="mt-3 text-slate-600">
        接下来我们会逐步搭建一个支持响应式和深色主题的学习仪表盘。
      </p>
    </main>
  );
}
```

运行：

```
npm run dev
```

标题应该明显变大并呈靛蓝色，页面背景应呈浅灰色。如果只是显示了文字、样式却完全没变化，请按顺序检查：CSS 入口是否被 main.jsx 引入；vite.config.js 是否注册 tailwindcss()；终端是否存在构建错误；是否误装了 v3 包或复制了 v3 配置。

## 2.7 浏览器兼容性提醒

Tailwind v4 面向现代浏览器。官方兼容性文档列出的核心要求是 Chrome 111+、Safari 16.4+ 和 Firefox 128+。如果项目必须支持更旧的浏览器，不要只为了使用新配置而直接升级；请先评估兼容性，并考虑继续使用 v3.4。浏览器目标属于项目技术决策，而不是样式偏好。

官方参考：

- https://tailwindcss.com/docs/installation/using-vite
- https://tailwindcss.com/docs/compatibility

### 本章练习

把标题改为 `text-4xl`，背景改为 `bg-slate-100`，再把文字颜色改为 `text-emerald-700`。每次只改一个类名，确认页面会按预期变化。

---

# 第 3 章：读懂 utility 类名并构建视觉基础

Tailwind 的类名通常有规律。初学者不必一次掌握每个 utility，但应先理解最常用的类别，以及它们如何组合成视觉层次。

## 3.1 显示方式与布局

| 常见类名 | 主要作用 | 典型用途 |
| --- | --- | --- |
| `block` | 块级显示 | 普通内容区块 |
| `inline` | 行内显示 | 不需要独立占一行的文本元素 |
| `inline-block` | 行内排列且可设置盒尺寸 | 小徽标 |
| `flex` | 建立 Flex 布局 | 水平导航、工具栏、按钮内容 |
| `inline-flex` | 行内 Flex 容器 | 图标加文字的按钮 |
| `grid` | 建立 Grid 布局 | 卡片列表、后台仪表盘 |
| `hidden` | `display: none` | 在特定断点隐藏内容 |
| `relative` | 相对定位 | 给内部绝对定位元素建立参照 |
| `absolute` | 绝对定位 | 角标、覆盖层 |
| `fixed` | 相对视口定位 | 固定操作按钮、弹出层 |
| `sticky` | 滚动到阈值时粘住 | 顶部栏、目录侧栏 |

注意 `hidden` 不等于“视觉上透明”。它会让元素不参与正常布局，而且屏幕阅读器一般也无法把它作为可见内容读取。因此不要用 `hidden` 来替代视觉隐藏或无障碍隐藏的概念。

## 3.2 间距：padding、margin 和 gap

Tailwind 使用许多带数字的间距 utility。常见例子：

```
<section class="p-6">
  <h2 class="mb-3">标题</h2>
  <div class="flex gap-4">
    <button class="px-4 py-2">取消</button>
    <button class="px-4 py-2">确认</button>
  </div>
</section>
```

- `p-6`：四周内边距。
- `px-4`：左右内边距；`py-2`：上下内边距。
- `mb-3`：下外边距。
- `gap-4`：Flex 或 Grid 子项之间的间距。

初学时，布局间隔优先考虑 gap-*，因为它表达的是“子项之间的距离”，通常比给每个子元素分别加 margin 更清楚。当然并不是所有情况都能用 gap 解决，单个元素与外部内容的距离仍可能需要 margin。

不要将 p-4 机械地理解为永远等于固定像素值。默认间距尺度与主题变量相关，项目也可以自定义尺度。最重要的是遵守团队统一的间距节奏，不要在同一个设计系统中任意组合很多近似但没有规则的间距。

## 3.3 宽高和尺寸边界

常见尺寸工具：

```
<div class="w-full max-w-xl">限制最大宽度的内容区</div>
<div class="min-h-screen">至少占满视口高度</div>
<div class="h-2 w-32">固定高度的进度条底槽</div>
<img class="h-12 w-12 rounded-full object-cover" src="avatar.jpg" alt="用户头像" />
```

- w-full：宽度占满父级可用空间。
- max-w-xl：设置最大宽度上限。
- min-h-screen：最小高度至少为视口高度。
- object-cover：图片铺满目标框，超出的部分裁切。
- size-12：当宽高相同，可以使用统一的尺寸 utility（支持该类名的版本中）。也可以分别写 h-12 w-12，让意图更显式。

容器宽度最常用的组合之一是：

```
<div class="mx-auto w-full max-w-6xl px-4 sm:px-6 lg:px-8">
  <!-- 页面内容 -->
</div>
```

它表示容器在水平方向居中、宽度不超出上限，并且屏幕越宽时内边距可以逐步增加。不要默认认为一个 container 类就会完成所有居中和留白需求；请明确表达 mx-auto、max-w-* 和 px-* 的职责。

## 3.4 排版：字号、粗细、行高与对齐

```
<h1 class="text-3xl font-bold tracking-tight">继续你的学习</h1>
<p class="mt-2 text-base leading-7 text-slate-600">这一段文字解释页面的主要目的。</p>
<span class="text-xs font-semibold uppercase tracking-wide">课程进度</span>
```

- text-3xl：字号层级。
- font-bold / font-semibold：字重。
- leading-7：行高。
- tracking-tight：字间距略紧。
- text-slate-600：文本颜色。
- text-center：文字居中。
- truncate：单行溢出时截断并显示省略号（使用前必须确认这是符合内容需求的行为）。

排版不只是字号。标题、正文、说明文字和错误提示应形成清晰层次。不要让所有文字都用不同颜色“抢注意力”。先确定语义层级，再给出合适的字号、粗细和行高。

## 3.5 颜色、边框、圆角与阴影

```
<article class="rounded-2xl border border-slate-200 bg-white p-6 shadow-sm">
  <h2 class="text-lg font-semibold text-slate-900">课程名称</h2>
  <p class="mt-2 text-sm text-slate-600">课程说明</p>
</article>
```

常见家族：

- 背景：bg-white、bg-slate-50、bg-indigo-600。
- 文字：text-slate-900、text-slate-600、text-white。
- 边框：border、border-slate-200。
- 圆角：rounded-md、rounded-lg、rounded-2xl、rounded-full。
- 阴影：shadow-sm、shadow-md、shadow-lg。
- 过渡：transition、duration-200。

这里要分清两种做法：学习或一次性原型可以先使用内置色彩；当一个项目开始需要统一品牌和主题时，应逐步把直接使用 indigo-600 之类的色彩类名替换为自己的语义 token。后面的主题章节会建立这套做法。

## 3.6 任意值应该何时用

如果设计确实需要一个不在默认尺度中的值，可以使用方括号形式的 arbitrary value，例如：

```
<div class="top-[5.25rem] w-[27rem]">精确的尺寸需求</div>
```

任意值不是错误；问题在于把每个尺寸都做成任意数值，会失去统一尺度，也更容易造成响应式问题。先使用默认尺度或项目 token；只有当值有明确设计理由时，再使用任意值。

### 本章练习：构建一张课程卡片

```
export function LessonCard() {
  return (
    <article className="rounded-2xl border border-slate-200 bg-white p-5 shadow-sm">
      <p className="text-xs font-semibold uppercase tracking-wide text-indigo-700">
        前端基础
      </p>
      <h2 className="mt-2 text-xl font-bold text-slate-900">
        理解 Flexbox 布局
      </h2>
      <p className="mt-2 leading-6 text-slate-600">
        学会排列、对齐与控制子项空间，让常见页面布局更容易维护。
      </p>
      <button className="mt-5 rounded-lg bg-indigo-600 px-4 py-2 font-semibold text-white hover:bg-indigo-700">
        继续学习
      </button>
    </article>
  );
}
```

请把边框、背景、内边距、字号、文字颜色、按钮交互状态逐项找出来。如果你能解释卡片的每一种视觉特征来自哪个 utility，就已经掌握了基本阅读方式。

---

# 第 4 章：真正理解 Flex、Grid、溢出与定位

响应式页面常常不是颜色出错，而是布局模型没选对。先理解 CSS 布局，再选择 Tailwind 类名。

## 4.1 Flexbox 适合一维排列

Flex 通常适合一个方向上的排列，例如导航项、按钮组、头像与文本、卡片头部。

```
<div class="flex items-center justify-between gap-4">
  <div class="flex min-w-0 items-center gap-3">
    <img class="size-10 rounded-full object-cover" src="avatar.jpg" alt="" />
    <div class="min-w-0">
      <p class="truncate font-semibold">课程负责人名称可能很长</p>
      <p class="text-sm text-slate-500">课程讲师</p>
    </div>
  </div>
  <button class="shrink-0 rounded-lg border px-3 py-2">查看</button>
</div>
```

关键类名：

- `flex`：启用 Flexbox。
- `items-center`：交叉轴对齐到中间。
- `justify-between`：主轴两侧分布。
- `gap-3`：子项之间间隔。
- `min-w-0`：允许该 flex 子项在必要时缩小到内容最小宽度以下，常用于避免长文本撑破容器。
- `truncate`：当内容确实应该单行省略时再使用。
- `shrink-0`：不让按钮或图标被挤压变窄。

Flex 的 `justify-*` 和 `items-*` 不是同一个方向。`justify-*` 对应主轴，主轴方向会受 `flex-row` 或 `flex-col` 影响。理解这一点，能减少大量试错。

## 4.2 Grid 适合二维布局

当内容同时有行和列时，Grid 通常更直接。例如卡片列表：

```
<div class="grid grid-cols-1 gap-4 sm:grid-cols-2 xl:grid-cols-3">
  <article class="rounded-xl border p-5">第一张卡片</article>
  <article class="rounded-xl border p-5">第二张卡片</article>
  <article class="rounded-xl border p-5">第三张卡片</article>
</div>
```

默认一列，达到 `sm` 断点后两列，达到 `xl` 后三列。注意不是因为小屏幕是一列，就需要另外写一套完全不同的 DOM；通常改变网格列数、方向与间距即可。

在高级布局中，你可能会遇到：

```
<main class="grid gap-8 lg:grid-cols-[minmax(0,1fr)_18rem]">
  <section>主要内容</section>
  <aside>侧边信息</aside>
</main>
```

这里的任意值网格轨道让主栏占据剩余空间，侧栏宽度为 `18rem`。`minmax(0,1fr)` 中的 `0` 可帮助主栏避免因为长内容的最小尺寸而把页面撑宽。

## 4.3 控制溢出，而不是把问题藏起来

常见 overflow utility：

- `overflow-hidden`：裁切超出盒子的内容。
- `overflow-auto`：有需要时出现滚动条。
- `overflow-x-auto`：只允许横向滚动。
- `overflow-y-auto`：只允许纵向滚动。
- `overflow-visible`：允许内容溢出默认盒子边界。

例如宽度很大的数据表可以使用：

```
<div class="overflow-x-auto rounded-xl border border-slate-200">
  <table class="min-w-[48rem] w-full text-left text-sm">
    <!-- 表头和表格内容 -->
  </table>
</div>
```

这会在较窄屏幕上允许表格容器水平滚动，而不是把整个页面撑出视口。不要默认给 `body` 加 `overflow-hidden` 来“解决”页面横向溢出，因为这可能把需要访问的内容直接裁掉。更好的做法是找到实际溢出的子元素，决定应该换行、缩小、截断还是局部滚动。

## 4.4 定位与 z-index

一个常见角标：

```
<div class="relative rounded-2xl border p-6">
  <span class="absolute right-3 top-3 rounded-full bg-emerald-100 px-2 py-1 text-xs font-semibold text-emerald-800">
    已完成
  </span>
  <h2 class="pr-20 text-lg font-semibold">完成课程基础部分</h2>
</div>
```

父元素 `relative` 建立定位参照；标签 `absolute` 放置在右上角；`pr-20` 给标签预留空间，避免标题被遮挡。单纯提高 `z-index` 并不能解决所有遮挡问题，因为 CSS 层叠上下文（stacking context）会影响层级关系。

常用定位 utility：

- `relative`：保持正常文档流，但作为定位参照。
- `absolute`：从正常文档流中移除并相对定位参照放置。
- `fixed`：通常相对视口定位。
- `sticky top-0`：滚动到顶部阈值后粘住。
- `z-10`、`z-20`、`z-50`：设置层级；具体值要结合层叠上下文来理解。

建立简单且有约定的层级体系，比所有弹层都写 `z-[99999]` 更可维护。例如，基础内容使用默认层级、顶部导航 `z-20`、模态框 `z-50`。真正项目应根据对话框、下拉菜单、提示气泡和固定导航定义层级规则。

## 4.5 容器宽度与移动端布局

很多页面可以从这个容器模式开始：

```
<div class="mx-auto w-full max-w-6xl px-4 sm:px-6 lg:px-8">
  <div class="grid grid-cols-1 gap-6 lg:grid-cols-[minmax(0,1fr)_18rem]">
```

```
    <section>主要内容</section>
    <aside>侧边栏</aside>
  </div>
</div>
```

外层负责页面宽度和左右留白；内层负责主内容与侧栏的关系。不要让同一个元素同时承担过多布局责任。当代码变复杂时，先看每层 DOM 是不是可以清楚地解释为容器、布局、区块或内容。

### 本章练习

1. 用 `flex` 让头像和两行用户文字水平排列。
2. 把卡片集合改成 `grid-cols-1 sm:grid-cols-2 xl:grid-cols-3`。
3. 给标题加入长文本，观察 `min-w-0`、`truncate` 和 `shrink-0` 分别解决什么问题。
4. 把一个绝对定位的角标放进 `relative` 容器，并预留标题空间。

---

# 第 5 章：响应式设计——从移动端开始，而不是最后补救

## 5.1 Tailwind 的断点是“从这里开始生效”

Tailwind 的响应式类默认采用 mobile-first 语义。无前缀的 utility 先应用；带 `sm:`、`md:` 等前缀的规则会从该断点及更宽的视口开始生效。

官方默认断点（v4 文档）：

| 前缀 | 最小宽度 | 大致理解 |
| --- | --- | --- |
| `sm:` | 40rem（通常约 640px） | 较宽手机或小平板 |
| `md:` | 48rem（通常约 768px） | 平板或较窄桌面 |
| `lg:` | 64rem（通常约 1024px） | 桌面布局常见阈值 |
| `xl:` | 80rem（通常约 1280px） | 宽屏 |
| `2xl:` | 96rem（通常约 1536px） | 更宽的桌面布局 |

`rem` 与根字号相关，因此不建议把断点粗暴地当成永远固定不变的像素。断点主要是一组默认设计尺度；项目可以根据产品实际需求自定义。

## 5.2 先写小屏默认值

假设卡片移动端一列、桌面端三列：

```
<div class="grid grid-cols-1 gap-4 md:grid-cols-2 xl:grid-cols-3">
  <!-- cards -->
</div>
```

不要写成想象中的“手机断点”。你不需要给每一种手机尺寸都写规则。更好的方式是先在最窄的布局下确保内容可读，然后只在版面确实需要变化时加断点。

## 5.3 断点可用于任何 utility

响应式不只是改列数。以下示例会改变标题尺寸、间距和排列方向：

```
<section class="px-4 py-6 sm:px-6 sm:py-8 lg:px-8 lg:py-10">
  <div class="flex flex-col gap-4 sm:flex-row sm:items-end sm:justify-between">
    <div>
      <p class="text-sm font-semibold text-indigo-700">学习空间</p>
      <h1 class="mt-2 text-2xl font-bold tracking-tight sm:text-3xl lg:text-4xl">
        继续你的学习
      </h1>
    </div>
    <button class="w-full rounded-lg bg-indigo-600 px-4 py-3 font-semibold text-white sm:w-auto">
      查看全部课程
    </button>
  </div>
</section>
```

这段代码中，小屏按钮是满宽，较大屏幕恢复为内容宽度；标题字号逐步放大；标题区块和操作按钮从上下排列变成左右排列。响应式规则全部留在元素附近，便于理解不同尺寸的变化。

## 5.4 如何决定断点

不要只因为设计稿上有“手机、平板、桌面”三类设备，就把断点固定成三类硬件。更实用的判断方式是：**当内容开始拥挤、行长不合理或布局无法继续工作时，才设置断点。**

举例：

- 导航项放不下时，考虑折叠菜单或换行策略。
- 三列卡片变得过窄时，改为两列或一列。
- 表单两列输入在窄屏中难以使用时，切换为一列。
- 页面主内容与侧栏在宽屏呈双栏，在窄屏应明确侧栏内容的顺序和优先级。

## 5.5 自定义断点

v4 中可通过 CSS-first 的 `@theme` 定义断点变量。例如：

```
@import "tailwindcss";

@theme {
  --breakpoint-xs: 30rem;
  --breakpoint-3xl: 120rem;
}
```

之后可使用 `xs:` 或 `3xl:`。定义自定义断点之前，先判断是否真的需要它。如果项目的每个页面都需要很多特殊断点，通常应重新审视设计系统，而不是继续增加断点数量。

## 5.6 容器查询的区别

视口断点依据整个浏览器宽度；容器查询可以根据某个父容器的宽度改变其子内容。它适合被复用在不同尺寸面板中的组件：同一张信息卡片在窄侧栏里纵向排列，在宽主栏里横向排列。

```
<div class="@container">
  <article class="flex flex-col gap-4 @md:flex-row @md:items-center">
    <div>封面或图标</div>
    <div>文字内容</div>
```

```
  </article>
</div>
```

使用容器查询前，应先确认目标元素是由父容器宽度决定布局，而不是全局视口宽度决定。两种模式解决的是不同问题。

官方参考：https://tailwindcss.com/docs/responsive-design

### 本章练习

为第 3 章的课程卡片集合制作三种尺寸：小屏一列、中屏两列、宽屏三列。再让按钮在小屏满宽、宽屏自动宽度。用浏览器开发者工具拖动视口，不只看一个固定桌面尺寸。

---

# 第 6 章：交互状态——hover、focus、active、disabled、group 与 peer

界面不是静态图片。真实用户会悬停、点击、通过键盘移动焦点，也会看到禁用、错误和选中状态。Tailwind 使用变体（variant）把这些状态附着在普通 utility 前面。

## 6.1 变体语法

最常见的形式是：

`状态前缀:工具类`

例如 `hover:bg-indigo-700` 表示“元素处于 hover 状态时，背景使用 indigo-700”。默认类提供基础样式，带前缀的类提供条件样式。

```
<button
  class="rounded-lg bg-indigo-600 px-4 py-2 font-semibold text-white transition hover:bg-indigo-700 active:bg-indigo-800 focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-indigo-600"
>
  保存
</button>
```

这里分别处理了基础背景、悬停、按下和键盘可见焦点。`transition` 可以让某些属性的变化更平滑，但过渡动画不是可访问性的替代品。

## 6.2 `hover:`、`focus:`、`focus-visible:` 和 `active:`

- `hover:`：指针悬停。触屏设备没有传统鼠标悬停，所以不能把关键操作只放在 hover 里。
- `focus:`：元素获得焦点时。它可能来自键盘，也可能来自其他焦点行为。
- `focus-visible:`：浏览器判断当前焦点应当被明显提示时，常用于提供清楚但不干扰鼠标使用的键盘焦点样式。
- `active:`：用户按下或激活元素的状态，常用于按钮即时反馈。

不要轻易使用 `outline-none` 然后什么都不补。焦点样式对键盘用户至关重要。如果去掉默认 outline，要明确提供足够对比的替代焦点样式。

## 6.3 disabled: 与禁用语义

```
<button
  disabled={isSaving}
  className="rounded-lg bg-indigo-600 px-4 py-2 font-semibold text-white hover:bg-indigo-700 disabled:cursor-not-allowed
disabled:opacity-50"
>
  {isSaving ? "正在保存…" : "保存更改"}
</button>
```

disabled: 只是根据 disabled 状态调整外观；真正让原生按钮进入禁用状态的是 JSX 的 disabled={isSaving} 属性。仅写 opacity-50 并不会阻止用户点击，也不会自动提供真实禁用语义。

对非原生按钮的自定义控件，通常需要更仔细地考虑 aria-disabled、键盘行为和事件处理。新手应尽量用原生的 <button>、<a>、<input>、<select> 等元素建立正确语义，再使用 Tailwind 调整它们的外观。

## 6.4 group-*：父元素状态影响子元素

当你希望悬停一张卡片时，卡片里的标题同时出现下划线或箭头移动，可以在父元素上加 group，然后在子元素上使用 group-hover:*。

```
<a href="/lesson/flexbox" class="group block rounded-2xl border border-slate-200 p-5 hover:border-indigo-300 hover:shadow-
md">
  <div class="flex items-center justify-between gap-4">
    <h2 class="font-semibold text-slate-900 group-hover:text-indigo-700">
      Flexbox 布局
    </h2>
    <span class="transition-transform group-hover:translate-x-1" aria-hidden="true">→</span>
  </div>
  <p class="mt-2 text-sm text-slate-600">把元素排列、对齐并控制空间。</p>
</a>
```

group 本身通常不会改变外观；它给后代元素提供一个状态参照。后代类名中的 group-hover: 才描述状态下的样式。

如果一个组件中有多个不同 group 作用域，Tailwind 支持为 group 命名，例如 group/item 和 group-hover/item:*，避免状态引用错误的祖先。

## 6.5 peer-*：相邻元素状态影响另一个元素

peer 常用于表单说明或错误消息，让后面的元素根据前面输入框的状态变化。

```
<form class="max-w-md">
  <label for="email" class="block text-sm font-medium text-slate-700">电子邮箱</label>
  <input
    id="email"
    type="email"
    required
    aria-describedby="email-help"
    class="peer mt-1 w-full rounded-lg border border-slate-300 px-3 py-2 focus-visible:outline-2 focus-visible:outline-
indigo-600 invalid:border-rose-500"
    placeholder="you@example.com"
  />
  <p id="email-help" class="mt-2 text-sm text-slate-500 peer-invalid:text-rose-600">
```

```
    请输入有效的电子邮箱地址。
  </p>
</form>
```

这里输入框是 `peer`，紧随其后的段落用 `peer-invalid:*` 反映输入框的无效状态。peer 关系通常依赖 CSS 可匹配的后续兄弟关系，所以元素顺序是有意义的；如果目标元素不是 peer 后面的兄弟元素，请考虑调整结构、用 group，或使用数据属性表达状态。

## 6.6 `aria-*`、`data-*` 与状态样式

交互状态不总是浏览器原生伪类。自定义弹窗可能有 `aria-expanded="true"`，标签页可能有 `data-selected="true"`。Tailwind 支持相关变体：

```
<button
  aria-expanded="true"
  class="rounded-lg px-3 py-2 aria-expanded:bg-indigo-50 aria-expanded:text-indigo-700"
>
  菜单
</button>

<div data-status="complete" class="rounded-lg data-[status=complete]:border-emerald-300 data-[status=complete]:bg-emerald-50">
  课程已完成
</div>
```

请记住：`aria-expanded` 必须由交互逻辑正确更新，`data-status` 也应该来自真正的应用状态。仅加一个属性并不自动实现菜单打开或业务状态改变。

## 6.7 组合多个变体

变体可以组合。例如：

```
<button class="bg-indigo-600 hover:bg-indigo-700 disabled:hover:bg-indigo-600 md:dark:hover:bg-indigo-500">
  按钮示例
</button>
```

但组合越复杂，越应该考虑职责是否清楚。若一个 className 中出现大量断点、主题、父子状态和业务状态，请把公共样式提取为组件，将真正变化的部分明确建模为 props 或状态，而不是继续扩大字符串。

官方参考：https://tailwindcss.com/docs/hover-focus-and-other-states

### 本章练习

1. 给按钮添加 `hover:`、`active:` 和 `focus-visible:` 状态。
2. 让卡片 hover 时改变标题与箭头，使用 `group`。
3. 给邮箱输入框增加 `required`，用 `peer-invalid:` 显示错误颜色。
4. 设置按钮 `disabled`，确认真实禁用和禁用外观同时存在。

---

# 第 7 章：深色模式、主题令牌与语义颜色

如果一个项目把 `bg-white`、`text-slate-900`、`border-slate-200` 散落在几十个组件中，初期很快，但主题调整时很难维护。成熟项目应该把颜色表达为设计系统中的语义：页面底色、卡片表面、主要文字、次要文字、边框、品牌主色等。

## 7.1 先理解“调色值”与“语义 token”不同

调色值表达视觉选择，例如 `indigo-600`；语义 token 表达用途，例如 `brand`、`canvas`、`surface`、`ink`、`muted`、`line`。

语义 token 的优势是，组件不需要关心具体的颜色值。品牌色从靛蓝调整为蓝色时，你更新主题；深色模式启用时，你调整 token 背后的值。组件仍然使用 `bg-brand`、`bg-surface` 和 `text-ink`，不需要每个文件逐个替换。

一份最小语义表可以是：

| token | 用途 | 浅色主题示例 | 深色主题示例 |
| --- | --- | --- | --- |
| `canvas` | 页面大背景 | 很浅的灰色 | 深蓝灰 |
| `surface` | 卡片、面板背景 | 白色 | 深灰蓝 |
| `ink` | 主文本 | 深色 | 接近白色 |
| `muted` | 次级文本 | 中灰 | 浅灰 |
| `line` | 边框、分隔线 | 浅灰 | 深灰 |
| `brand` | 品牌操作与链接 | 品牌主色 | 为暗背景调整过的品牌色 |
| `brand-hover` | 品牌按钮 hover | 较深主色 | 适用于暗主题的 hover 色 |

颜色值只是示例，正式产品需要根据品牌规范、对比度和视觉测试来决定。

## 7.2 v4 的 CSS-first 主题配置

Tailwind v4 把许多主题配置直接放进 CSS 的 `@theme` 中。`@theme` 不只是普通 CSS 自定义属性声明；它告诉 Tailwind 这些主题变量应该参与 utility 类名的生成。

我们为课程学习仪表盘建立一个主题。把 `src/index.css` 更新为：

```
@import "tailwindcss";

/* 让 dark: 变体在 html 或祖先带有 .dark 类时生效 */
@custom-variant dark (&:where(.dark, .dark *));

@theme inline {
  --color-canvas: var(--canvas);
  --color-surface: var(--surface);
  --color-ink: var(--ink);
  --color-muted: var(--muted);
  --color-line: var(--line);
  --color-brand: var(--brand);
  --color-brand-hover: var(--brand-hover);
  --color-on-brand: var(--on-brand);
  --font-sans: var(--font-ui);
```

```
}

:root {
  color-scheme: light;
  --canvas: #f8fafc;
  --surface: #ffffff;
  --ink: #0f172a;
  --muted: #475569;
  --line: #e2e8f0;
  --brand: #4f46e5;
  --brand-hover: #4338ca;
  --on-brand: #ffffff;
  --font-ui: Inter, ui-sans-serif, system-ui, sans-serif;
}

.dark {
  color-scheme: dark;
  --canvas: #0b1220;
  --surface: #111827;
  --ink: #e5e7eb;
  --muted: #a1a1aa;
  --line: #293548;
  --brand: #a5b4fc;
  --brand-hover: #c7d2fe;
  --on-brand: #1e1b4b;
}

@layer base {
  html {
    min-width: 320px;
  }

  body {
    margin: 0;
    background: var(--canvas);
    color: var(--ink);
    font-family: var(--font-ui);
  }
}
```

现在 Tailwind 可以根据 `--color-canvas` 等主题变量生成 `bg-canvas`、`text-ink`、`border-line`、`bg-brand` 等类名。因为用的是 `@theme inline` 把 utility token 映射到普通 CSS 变量，浅色和深色主题切换时只需修改 `--canvas`、`--surface` 等变量即可。

请注意 `@custom-variant dark`：它建立的是手动 `.dark` 类驱动的暗色变体方式。与只跟随系统设置的策略不同，应用现在可以通过 JavaScript 切换根元素上的 `dark` 类。`dark:` 仍然是有用的；如果某些外观只在暗色模式变化，可以使用 `dark:*`。不过对普通背景、文字和边框而言，语义 token 自动切换通常更清楚，不必把每个类都写成 `bg-white dark:bg-slate-900`。

## 7.3 组件如何使用语义 token

之前卡片使用硬编码色彩：

```
<article className="border border-slate-200 bg-white text-slate-900">
  ...
</article>
```

改为：

```
<article className="border border-line bg-surface text-ink">
  <p className="text-muted">课程说明</p>
</article>
```

语义 token 并不会自动让每一种颜色选择都正确，但会让主题调整的影响范围更小、更容易测试。组件表达的是内容层级和意图，设计系统负责将意图映射为具体颜色。

## 7.4 什么时候仍然需要 dark:

如果所有内容只需替换基础表面色，token 是最佳入口。如果某个元素的样式本身在暗色模式需要改变，比如暗色模式里隐藏装饰性光晕、调整图片滤镜或改变阴影策略，可以显式使用：

```
<div class="shadow-md dark:shadow-none">
  内容
</div>
```

主题 token 与 `dark:` 并不互斥。前者解决语义颜色与主题值的集中管理，后者表达“这个特定 utility 在暗色模式下需要不同结果”。

## 7.5 深色模式开关

后面的完整项目会给出可以运行的 React 开关。核心逻辑是切换根 HTML 元素上的类：

```
function toggleTheme() {
  document.documentElement.classList.toggle("dark");
}
```

实际组件还应记录当前状态，更新按钮名称和 `aria-pressed`。如果产品要求记住用户的选择，可以把偏好写入 `localStorage`，并在页面初次加载前恢复。但要考虑系统偏好、手动选择和首屏闪烁之间的关系，不要把单行 toggle 当成完整的主题偏好系统。

## 7.6 用 @theme 自定义设计尺度

除颜色外，`@theme` 也能定义字体、间距、断点、阴影等主题变量。当你添加自定义颜色变量 `--color-brand` 时，就会得到对应的 `bg-brand`、`text-brand` 等 utility；添加 `--font-display` 可以使对应的字体 utility 成为设计系统的一部分。

不要把每一个一次性用到的值都提升为全局 token。通常只有那些反复出现、有语义用途、需要集中维护的值才值得成为主题变量。

官方参考：

- https://tailwindcss.com/docs/theme
- https://tailwindcss.com/docs/dark-mode

### 本章练习

1. 用 `@theme` 和 CSS 变量建立 `canvas`、`surface`、`ink`、`muted`、`line`、`brand`。
2. 让卡片只使用语义 token，而不直接写 `white` 或 `slate-900`。

3. 在根元素上切换 `.dark`，确认页面、卡片、边框和文字同时变化。
4. 检查深色主题中次要文字是否仍然清晰，不能只满足“变量能切换”。

# 第 8 章：可维护的 className、组件 API 与样式职责

Utility-first 会让简单组件的样式非常直观，但项目扩大后，需要用组件 API 把重复模式组织起来。抽象的目的不是消灭 className，而是让团队能用一致的方式表达组件外观和状态。

## 8.1 什么时候应该抽组件

假设应用中有四个按钮，它们都需要圆角、相同的字号、焦点状态和禁用样式，只是外观类型有所区别。复制四份长 className 容易出现不一致。此时应抽象一个 `Button` 组件，并让 `variant` 表达有意义的设计选择。

但如果一个区域只出现一次、语义简单、样式没有复用价值，过早创建十层组件反而会提高理解成本。通常可以按以下问题判断：

- 相同的视觉与交互规则是否在多个地方重复出现？
- 是否存在清晰、稳定的组件 API（例如 `variant`、`size`、`disabled`）？
- 抽象后是否更容易保证一致性和测试状态？
- 是否仍然可以通过 props 看懂组件用途，而不需要猜内部 CSS？

## 8.2 给组件定义样式职责

推荐一个简单边界：

- **全局 CSS / theme：** 设计令牌、默认字体、页面基础背景、浏览器 reset 相关规则。
- **通用组件：** 按钮、输入框、徽标、卡片等重复使用的视觉与交互模式。
- **页面组合：** 页面网格、区域间距、内容宽度、布局切换。
- **业务逻辑：** 当前状态、权限、加载状态、请求结果、用户是否完成课程。

避免把“这个按钮是否禁用”写成纯样式问题。禁用状态应由业务逻辑和原生 `disabled` 属性控制；Tailwind 只负责相应的外观反馈。

## 8.3 用明确映射管理变体

不要根据用户传入的字符串拼出部分类名。使用完整、静态的映射：

```
const buttonVariants = {
  primary: "bg-brand text-on-brand hover:bg-brand-hover",
  secondary: "border border-line bg-surface text-ink hover:bg-canvas",
  danger: "bg-rose-600 text-white hover:bg-rose-700",
};

function ExampleButton({ variant = "primary", children }) {
  return (
    <button className={`rounded-lg px-4 py-2 font-semibold ${buttonVariants[variant]}`}>
      {children}
    </button>
  );
}
```

在这个例子中，每个类名都以完整字符串存在，因此 Tailwind 能扫描到所有可能的变体。后续会在完整项目中把它升级成可接受 `className` 和原生 button props 的通用组件。

## 8.4 条件样式组合

简单场景可以使用模板字符串：

```
<button className={`rounded-lg px-4 py-2 ${isActive ? "bg-brand text-white" : "bg-surface text-ink"}`}>
  切换选项
</button>
```

更复杂场景可以使用 `clsx` 做条件组合，再使用 `tailwind-merge` 处理冲突的 Tailwind 类名：

```
npm install clsx tailwind-merge
```

创建 `src/lib/cn.js`：

```
import { clsx } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs) {
  return twMerge(clsx(inputs));
}
```

`clsx` 负责把字符串、数组或条件对象合并成 className；`twMerge` 识别一部分相互冲突的 Tailwind 类名，并按约定留下最终结果。它们不会替代设计决策：如果两个类名意图冲突，应该先弄清楚哪一个样式应该负责，而不是指望合并工具替你解决所有 CSS 级联问题。

## 8.5 创建可维护的 Button 组件

`src/components/Button.jsx`：

```
import { cn } from "../lib/cn.js";

const baseClasses =
  "inline-flex items-center justify-center gap-2 rounded-xl px-4 py-2.5 text-sm font-semibold transition focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-brand disabled:pointer-events-none disabled:opacity-50";

const variants = {
  primary: "bg-brand text-on-brand hover:bg-brand-hover active:translate-y-px",
  secondary: "border border-line bg-surface text-ink hover:bg-canvas",
  ghost: "text-muted hover:bg-canvas hover:text-ink",
};

export function Button({
  variant = "primary",
  className,
  type = "button",
  ...props
}) {
  const variantClasses = variants[variant] ?? variants.primary;
```

```
  return (
    <button
      type={type}
      className={cn(baseClasses, variantClasses, className)}
      {...props}
    />
  );
}
```

这个组件有几个关键点：

- 公共的形状、尺寸、焦点和禁用外观放在 baseClasses。
- variant 表达设计系统提供的外观选择。
- type 默认是 button，避免放在表单里时意外触发表单提交。
- ...props 让调用方仍然能够传入 disabled、onClick、aria-* 等原生属性。
- className 保留适度扩展空间；cn() 处理条件与常见类名冲突。

不要把每个 CSS 属性都变成一个独立 prop（例如 buttonBgColor、buttonRadiusValue、buttonFontSize）。组件 API 应表达对调用者有意义的变化，而不是复刻所有 CSS 属性。需要自由度过高时，考虑这是通用基础组件，还是只属于某个页面的特殊按钮。

## 8.6 状态与表现分离

看这个例子：

```
<Button disabled={isSaving}>
  {isSaving ? "正在保存…" : "保存更改"}
</Button>
```

React 状态决定文案和真实 disabled 属性；Button 组件只规定禁用时如何显示。这样业务逻辑和样式职责不会混在一个庞大的 className 条件中。

### 本章练习

1. 创建一个 Button，支持 primary、secondary 两种变体。
2. 让 disabled 控制真实禁用，并用 disabled: 变体调整外观。
3. 用 className 允许调用处添加必要的布局类，例如 w-full。
4. 不要用 bg-${color}-600 之类的字符串拼接；改用完整类名映射。

---

# 第 9 章：类名扫描机制、动态类名陷阱与 safelist

这是实际项目中很常见的一类 bug：代码看起来已经把类名拼出来了，但页面并没有对应样式。

## 9.1 Tailwind 不会运行你的 JavaScript 来找类名

Tailwind 将源文件当作普通文本扫描，并不完整解析每段 JS/TS 的运行时逻辑。例如下面代码有风险：

```
function Badge({ color }) {
  return <span className={`bg-${color}-600 text-white`}>状态</span>;
}
```

当 color="green" 时，浏览器运行时可能产生 bg-green-600，但源码中没有完整的 bg-green-600 字符串。Tailwind 无法通过运行这个组件来推断所有 color 可能取值，因此可能不会生成对应 CSS。

## 9.2 用完整的静态映射修复

```
const badgeClasses = {
  success: "bg-emerald-100 text-emerald-800",
  warning: "bg-amber-100 text-amber-800",
  danger: "bg-rose-100 text-rose-800",
};

function Badge({ status = "success", children }) {
  const classes = badgeClasses[status] ?? badgeClasses.success;

  return (
    <span className={`rounded-full px-2.5 py-1 text-xs font-semibold ${classes}`}>
      {children}
    </span>
  );
}
```

所有类名都是完整的字符串。即使某个状态在开发时还没出现，静态映射仍能被扫描到。对于来自数据库的业务状态，应该先把业务值映射到已知设计变体，再输出类名，而不是让服务端数据直接决定任意 className。

## 9.3 动态数值并不都需要动态类名

如果一个进度条的宽度来自服务端数值，不要拼出 w-${progress}%。宽度是连续的运行时数据，更适合使用受控的 inline style 或 CSS 自定义属性：

```
function ProgressBar({ value }) {
  const safeValue = Math.min(100, Math.max(0, value));

  return (
    <div
      className="h-2 overflow-hidden rounded-full bg-line"
      role="progressbar"
      aria-label="课程完成度"
      aria-valuemin={0}
      aria-valuemax={100}
      aria-valuenow={safeValue}
    >
      <span
        className="block h-full rounded-full bg-brand transition-[width]"
        style={{ width: `${safeValue}%` }}
      />
    </div>
  );
}
```

这里的 className 是固定的，而 width 是来自数值的样式属性。使用 inline style 并不意味着违背 utility-first；当值来自连续动态数据时，它可能是合适的表达方式。重点是不要生成工具无法识别的 utility 名称。

## 9.4 如果某些类名在源码中不可见怎么办

Tailwind v4 提供 `@source` 指令控制扫描来源和明确生成的 utility。

如果某些类名来自默认不会扫描的外部组件库，可以在 CSS 中登记来源路径（路径相对于当前 CSS 文件）：

```
@import "tailwindcss";
@source "../node_modules/@acmecorp/ui";
```

如果确实有一些 class 不会自然地出现在可扫描的源码里，可以使用 `@source inline()` 显式要求生成：

```
@import "tailwindcss";
@source inline("underline");
@source inline("{hover:,focus:,}underline");
```

第二行会要求生成基础、hover 和 focus 版本。范围较大的候选集可以使用 brace expansion，但要有意识地控制规模，避免为了保险而生成大批不需要的 CSS。

注意：v4 用的是 `@source inline()` 这套方式，不要把 v3 JavaScript 配置里的 `safelist` 直接视为 v4 默认可用配置选项。

## 9.5 v3 的内容扫描是另一套配置

在 Tailwind v3 中，一般需要在 `tailwind.config.js` 的 `content` 中配置要扫描的文件路径：

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ["./index.html", "./src/**/*.{js,ts,jsx,tsx}"],
  theme: { extend: {} },
  plugins: [],
};
```

v4 的默认检测能力和 CSS `@source` 不要同 v3 的 `content`/`safelist` 配置混用。维护旧项目时，应当根据该项目锁定的 Tailwind 版本阅读对应文档。

官方参考：https://tailwindcss.com/docs/detecting-classes-in-source-files

### 本章练习

1. 写一个会拼接 `text-${color}-600` 的组件，解释构建工具为什么可能漏掉类名。
2. 改成完整映射，并确认所有变体均能显示。
3. 把动态百分比用于进度条宽度，而不是构建 `w-*` 类名。
4. 阅读自己的源码，搜索 `bg-${`、`text-${`、`w-${` 这类模式，判断是否存在同类问题。

---

# 第 10 章：Tailwind v4 与 v3 的配置差异

这是避免“为什么官方教程和我项目不一样”的关键章节。很多教程仍然使用 Tailwind v3 的写法。它们可能完全正确，但不一定适用于 v4。

## 10.1 v4 的 CSS-first 配置

一个最小 v4 CSS 入口通常从下面开始：

```
@import "tailwindcss";
```

主题通过 CSS 中的 `@theme` 定义；扫描可通过 `@source` 补充；自定义变体可通过 `@custom-variant` 表达；自定义 utility 可用 `@utility` 定义。

示例：

```
@import "tailwindcss";

@theme {
  --color-brand: oklch(0.55 0.22 270);
  --font-display: "Inter", ui-sans-serif, system-ui, sans-serif;
  --breakpoint-3xl: 120rem;
}

@utility content-auto {
  content-visibility: auto;
}
```

定义了 `--color-brand` 后可以使用 `bg-brand`、`text-brand` 等对应 utility。`@utility` 用于注册 Tailwind 能够识别和组合的自定义 utility。普通 CSS 依然可以存在；只有在希望它成为可组合 utility 体系的一部分时，才需要自定义 utility 指令。

## 10.2 v3 的 JavaScript 配置方式

v3 项目常见的 CSS 入口是：

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

并且通过 `tailwind.config.js` 定义扫描路径、主题扩展和插件：

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ["./index.html", "./src/**/*.{js,ts,jsx,tsx}"],
  theme: {
    extend: {
      colors: {
        brand: "#4f46e5",
      },
    },
  },
  plugins: [],
};
```

这并不代表 v3 是坏的。对于已经上线且稳定运行的 v3 项目，应当保持相应配置并按版本文档维护，而不是随意复制 v4 的 CSS 规则进去。

## 10.3 差异对照

| 主题 | Tailwind v4 新项目 | Tailwind v3 常见方式 |
| --- | --- | --- |
| 主要 CSS 引入 | `@import "tailwindcss";` | `@tailwind base/components/utilities;` |
| Vite 集成 | `@tailwindcss/vite` 插件 | 常见做法是 PostCSS 集成 |

| 主题 | Tailwind v4 新项目 | Tailwind v3 常见方式 |
| --- | --- | --- |
| 主题 token | CSS `@theme` | `tailwind.config.js` 的 `theme` / `extend` |
| 扫描补充 | 自动检测，并可用 CSS `@source` | `content` 配置数组 |
| 显式生成候选类 | `@source inline()` | 配置中的 `safelist` |
| JavaScript 配置 | 仍可用于兼容场景，但不再自动发现 | `tailwind.config.js` 是常见中心配置 |
| 浏览器目标 | 现代浏览器要求较新 | v3 适用于更多较旧的项目环境，具体看版本与项目需求 |

### 10.4 v4 还支持 JavaScript 配置吗

支持向后兼容，但 v4 不再自动发现旧 JavaScript 配置文件。如果项目仍需加载它，可以通过 CSS 中的 `@config` 显式指向，例如：

```
@import "tailwindcss";
@config "../../tailwind.config.js";
```

路径要根据当前 CSS 文件位置和项目结构修正。官方升级文档还指出，v4 不支持旧配置中的 `corePlugins`、`safelist` 和 `separator` 等选项；对 safelist 类需求，应迁移为 `@source inline()`。所以“加了旧 config 文件但设置没生效”不一定是拼写问题，也可能是配置没有加载或选项已发生版本变化。

### 10.5 如何安全地选择版本

对于新手新项目：

1. 先选择目标主版本（本教程采用 v4）。
2. 安装该版本对应的集成插件。
3. 用同一主版本官方文档完成初次设置。
4. 锁定依赖并提交锁文件。
5. 升级前阅读官方 Upgrade Guide，检查浏览器兼容与 breaking changes。

对于旧项目：

1. 先查看 `package.json` 和锁文件确定正在使用的版本。
2. 按这个版本的文档修复问题，不要先将代码迁移到新版本语法。
3. 如果要升级，安排一次有测试和视觉回归的迁移，而不是把 v3 与 v4 的配置拼在一起。

官方参考：

- https://tailwindcss.com/docs/upgrade-guide
- https://v3.tailwindcss.com/docs/configuration
- https://v3.tailwindcss.com/docs/content-configuration

---

# 第 11 章：基础 CSS、层叠模型与可访问性仍然重要

Utility-first 能够降低样式抽象成本，但不能免除 CSS 的基本知识。理解以下内容，你才有能力诊断复杂布局与冲突。

## 11.1 盒模型和宽度计算

元素并不只有内容区。它还可能包含 padding、border 和 margin。Tailwind 默认提供合理的盒尺寸基础，但你仍需理解 `box-sizing`、`min-width`、`max-width`、`flex-shrink` 等属性的关系。

例如，某个容器设置了 `w-full`，内部又塞入一个不能缩小的子项，页面仍可能横向溢出。解决方案不是重复给父元素加 `w-full`，而是检查子项的最小尺寸、文本是否允许换行、是否应该使用 `min-w-0` 或局部 `overflow-x-auto`。

## 11.2 CSS 层叠和优先级

CSS 最终会经过来源、级联层、重要性、选择器优先级以及源代码顺序等规则来决定哪条声明生效。HTML 的 class 字符串中谁先谁后，并不简单等同于 CSS 最终谁覆盖谁。

例如，`class="p-4 p-8"` 不应被当作“后写的类必然生效”的可靠写法。框架生成的 CSS 顺序和冲突语义才决定结果。应避免在同一个元素上留下相互冲突的 utility；如确实需要允许调用方覆盖默认类，可以考虑使用 `tailwind-merge`，但仍应先有清楚的组件 API。

## 11.3 Preflight 和浏览器默认样式

Tailwind 导入会包含基础样式重置（Preflight）。这有助于减少不同浏览器默认样式造成的差异，但也意味着不能假设浏览器会给 `h1`、`p`、`button` 等元素设置自己习惯中的默认 margin 或标题大小。

因此，要显式给标题定义字号和间距，给按钮定义交互样式，给表单控件安排边框和焦点。Preflight 并不替你完成页面设计，也不意味着某个元素拥有正确的语义。

## 11.4 语义 HTML 先于视觉样式

- 用 `<button>` 表达可执行动作，用 `<a href="...">` 表达导航链接。
- 表单输入要有可关联的 `<label>`。
- 图片要根据用途填写适当的 `alt`；纯装饰图片才适合空 alt。
- 标题层级应该符合内容结构，不要只为了字号漂亮而跳过语义层级。
- 菜单、对话框、标签页等复杂交互，不只是添加 `hover:` 和 `group-*`，还需要管理焦点、键盘和 ARIA 状态。

## 11.5 焦点、对比度和状态可见性

所有关键交互都应该能通过键盘使用，并有可见焦点。主文字、次级文字、按钮文字、边框和错误提示应在浅色与深色主题中都保持足够可辨识度。不要仅通过颜色区分成功与失败，也应提供文案或图标等辅助信息。

一个可访问的按钮，不只是这样：

```
<button class="bg-indigo-600 text-white">提交</button>
```

还应该考虑交互反馈、键盘焦点、禁用状态、加载状态和按钮是否具有正确语义。视觉样式只占其中一部分。

## 11.6 何时写普通 CSS，何时写 utility

建议的判断方式：

- 单个页面的布局和常见视觉属性：优先 utility。
- 重复使用的按钮、卡片、徽标模式：先考虑组件 API，再决定是否需要 CSS 类。
- 设计 token 和全局默认行为：使用主题变量或基础 CSS。

- 框架未提供且具有通用复用价值的低层样式能力：考虑 `@utility`。
- 非常特殊、复杂且难以通过 utility 清楚表达的规则：可以写普通 CSS，并明确放置位置与层叠关系。

不要为了“百分百 Tailwind”把复杂 CSS 生硬地塞进 className，也不要因为 className 变长就马上把每个元素都转换成一个自定义 CSS 组件类。核心是让样式意图可读、可测试、可维护。

官方参考：https://tailwindcss.com/docs/adding-custom-styles

---

# 第 12 章：完整实践——做一个响应式学习仪表盘

前面章节的知识会在这里组合起来。这个小项目包含：语义颜色 token、明暗主题切换、移动端布局、响应式卡片 Grid、状态徽标、进度条、按钮组件、可访问性标签和静态 className 映射。

## 12.1 项目文件结构

确保项目中至少有这些文件：

```
src/
  components/
    Button.jsx
  lib/
    cn.js
  App.jsx
  index.css
  main.jsx
vite.config.js
```

前面第 2 章已配置好 `vite.config.js` 与 `main.jsx`。如果没有安装条件 className 合并工具，请执行：

```
npm install clsx tailwind-merge
```

## 12.2 src/index.css

使用第 7 章中完整的 CSS-first 主题配置。这是项目的样式基线：浅色和深色主题中的页面、表面、文字、边框和品牌色都从 token 读取。

```
@import "tailwindcss";

@custom-variant dark (&:where(.dark, .dark *));

@theme inline {
  --color-canvas: var(--canvas);
  --color-surface: var(--surface);
  --color-ink: var(--ink);
  --color-muted: var(--muted);
  --color-line: var(--line);
  --color-brand: var(--brand);
  --color-brand-hover: var(--brand-hover);
  --color-on-brand: var(--on-brand);
  --color-success-bg: var(--success-bg);
  --color-success-ink: var(--success-ink);
```

```
  --color-warning-bg: var(--warning-bg);
  --color-warning-ink: var(--warning-ink);
  --font-sans: var(--font-ui);
}

:root {
  color-scheme: light;
  --canvas: #f8fafc;
  --surface: #ffffff;
  --ink: #0f172a;
  --muted: #475569;
  --line: #e2e8f0;
  --brand: #4f46e5;
  --brand-hover: #4338ca;
  --on-brand: #ffffff;
  --success-bg: #dcfce7;
  --success-ink: #166534;
  --warning-bg: #fef3c7;
  --warning-ink: #92400e;
  --font-ui: Inter, ui-sans-serif, system-ui, sans-serif;
}

.dark {
  color-scheme: dark;
  --canvas: #0b1220;
  --surface: #111827;
  --ink: #e5e7eb;
  --muted: #a1a1aa;
  --line: #293548;
  --brand: #a5b4fc;
  --brand-hover: #c7d2fe;
  --on-brand: #1e1b4b;
  --success-bg: #123522;
  --success-ink: #bbf7d0;
  --warning-bg: #422006;
  --warning-ink: #fde68a;
}

@layer base {
  html {
    min-width: 320px;
  }

  body {
    margin: 0;
    background: var(--canvas);
    color: var(--ink);
    font-family: var(--font-ui);
  }
}
```

本例把成功和待开始等状态颜色也抽成了 token。原因是徽标是重复使用的业务 UI，如果直接在页面到处写浅绿、深绿和浅黄，深色模式以及品牌调整会很难统一。

### 12.3 src/lib/cn.js

```
import { clsx } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs) {
  return twMerge(clsx(inputs));
}
```

### 12.4 src/components/Button.jsx

```
import { cn } from "../lib/cn.js";

const baseClasses =
  "inline-flex items-center justify-center gap-2 rounded-xl px-4 py-2.5 text-sm font-semibold transition focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-brand disabled:pointer-events-none disabled:opacity-50";

const variants = {
  primary: "bg-brand text-on-brand hover:bg-brand-hover active:translate-y-px",
  secondary: "border border-line bg-surface text-ink hover:bg-canvas",
  ghost: "text-muted hover:bg-canvas hover:text-ink",
};

export function Button({
  variant = "primary",
  className,
  type = "button",
  ...props
}) {
  const variantClasses = variants[variant] ?? variants.primary;

  return (
    <button
      type={type}
      className={cn(baseClasses, variantClasses, className)}
      {...props}
    />
  );
}
```

### 12.5 src/App.jsx

```
import { useState } from "react";
import { Button } from "./components/Button.jsx";

const lessons = [
  {
    id: 1,
```

```
    title: "理解 Flexbox 布局",
    category: "CSS 布局",
    description: "从主轴、交叉轴到对齐规则，构建稳定的一维布局。",
    progress: 72,
    status: "in-progress",
    duration: "18 分钟",
  },
  {
    id: 2,
    title: "用 Grid 构建卡片系统",
    category: "CSS 布局",
    description: "学习如何用行与列组织仪表盘和内容集合。",
    progress: 35,
    status: "in-progress",
    duration: "24 分钟",
  },
  {
    id: 3,
    title: "让界面适应不同屏幕",
    category: "响应式设计",
    description: "根据内容需要设置断点，而不是只照顾某一台设备。",
    progress: 100,
    status: "complete",
    duration: "16 分钟",
  },
];

const statusClasses = {
  "in-progress": "bg-warning-bg text-warning-ink",
  complete: "bg-success-bg text-success-ink",
};

const statusLabels = {
  "in-progress": "学习中",
  complete: "已完成",
};

function ProgressBar({ value, label }) {
  const safeValue = Math.min(100, Math.max(0, value));

  return (
    <div
      className="h-2 overflow-hidden rounded-full bg-line"
      role="progressbar"
      aria-label={label}
      aria-valuemin={0}
      aria-valuemax={100}
      aria-valuenow={safeValue}
    >
      <span
```

```
        className="block h-full rounded-full bg-brand transition-[width] duration-300"
        style={{ width: `${safeValue}%` }}
      />
    </div>
  );
}

function LessonCard({ lesson }) {
  const statusClass = statusClasses[lesson.status] ?? statusClasses["in-progress"];
  const statusLabel = statusLabels[lesson.status] ?? "学习中";

  return (
    <article id={`lesson-${lesson.id}`} className="group flex h-full flex-col rounded-2xl border border-line bg-surface p-5
shadow-sm transition hover:-translate-y-0.5 hover:shadow-md">
      <div className="flex items-start justify-between gap-3">
        <p className="text-xs font-semibold uppercase tracking-wide text-brand">
          {lesson.category}
        </p>
        <span className={`shrink-0 rounded-full px-2.5 py-1 text-xs font-semibold ${statusClass}`}>
          {statusLabel}
        </span>
      </div>

      <h2 className="mt-4 text-lg font-bold tracking-tight text-ink group-hover:text-brand">
        {lesson.title}
      </h2>
      <p className="mt-2 flex-1 text-sm leading-6 text-muted">
        {lesson.description}
      </p>

      <div className="mt-5 flex items-center justify-between gap-3 text-xs text-muted">
        <span>学习进度</span>
        <span className="font-semibold tabular-nums">{lesson.progress}%</span>
      </div>
      <div className="mt-2">
        <ProgressBar value={lesson.progress} label={`${lesson.title}的学习进度`} />
      </div>

      <div className="mt-5 flex items-center justify-between gap-3 border-t border-line pt-4">
        <span className="text-xs text-muted">{lesson.duration}</span>
        <a
          href={`#lesson-${lesson.id}`}
          className="inline-flex items-center gap-1 text-sm font-semibold text-brand focus-visible:outline-2 focus-
visible:outline-offset-2 focus-visible:outline-brand"
        >
          {lesson.status === "complete" ? "复习课程" : "继续学习"}
          <span className="transition-transform group-hover:translate-x-1" aria-hidden="true">→</span>
        </a>
      </div>
    </article>
```

```
  );
}

export default function App() {
  const [isDark, setIsDark] = useState(() =>
    document.documentElement.classList.contains("dark"),
  );

  function handleThemeToggle() {
    const next = !isDark;
    document.documentElement.classList.toggle("dark", next);
    setIsDark(next);
  }

  const completedCount = lessons.filter((lesson) => lesson.status === "complete").length;
  const overallProgress = Math.round(
    lessons.reduce((sum, lesson) => sum + lesson.progress, 0) / lessons.length,
  );

  return (
    <div className="min-h-screen bg-canvas text-ink">
      <header className="sticky top-0 z-20 border-b border-line bg-surface/90 backdrop-blur">
        <div className="mx-auto flex max-w-6xl items-center justify-between gap-4 px-4 py-4 sm:px-6 lg:px-8">
          <a href="#top" className="flex min-w-0 items-center gap-3 rounded-lg focus-visible:outline-2 focus-
visible:outline-offset-2 focus-visible:outline-brand">
            <span className="flex size-10 shrink-0 items-center justify-center rounded-xl bg-brand text-lg font-black text-
on-brand" aria-hidden="true">
              L
            </span>
            <span className="truncate text-base font-bold tracking-tight">学习空间</span>
          </a>

          <div className="flex items-center gap-2 sm:gap-3">
            <span className="hidden text-sm text-muted sm:inline">本周学习计划</span>
            <Button
              variant="secondary"
              onClick={handleThemeToggle}
              aria-pressed={isDark}
              aria-label={isDark ? "切换到浅色主题" : "切换到深色主题"}
            >
              {isDark ? "浅色模式" : "深色模式"}
            </Button>
          </div>
        </div>
      </header>

      <main id="top" className="mx-auto grid max-w-6xl gap-8 px-4 py-8 sm:px-6 lg:grid-cols-[minmax(0,1fr)_18rem] lg:px-8
lg:py-10">
        <section>
          <p className="text-sm font-semibold text-brand">你的学习进度</p>
```

```
  <div className="mt-2 flex flex-col gap-4 sm:flex-row sm:items-end sm:justify-between">
    <div>
      <h1 className="text-3xl font-bold tracking-tight sm:text-4xl">继续你的学习</h1>
      <p className="mt-3 max-w-2xl leading-7 text-muted">
        每次完成一个小步骤，逐渐建立对布局、主题和组件状态的完整理解。
      </p>
    </div>
    <Button className="w-full sm:w-auto" onClick={() => window.location.hash = "lesson-1"}>
      继续上次学习 <span aria-hidden="true">→</span>
    </Button>
  </div>

  <div className="mt-8 grid grid-cols-1 gap-4 sm:grid-cols-2 xl:grid-cols-3">
    {lessons.map((lesson) => (
      <LessonCard key={lesson.id} lesson={lesson} />
    ))}
  </div>
</section>

<aside className="space-y-5">
  <section className="rounded-2xl border border-line bg-surface p-5">
    <h2 className="text-base font-bold">学习概况</h2>
    <p className="mt-1 text-sm text-muted">当前演示数据汇总</p>

    <div className="mt-5 grid grid-cols-2 gap-3">
      <div className="rounded-xl bg-canvas p-4">
        <p className="text-xs text-muted">课程总数</p>
        <p className="mt-1 text-2xl font-bold tabular-nums">{lessons.length}</p>
      </div>
      <div className="rounded-xl bg-canvas p-4">
        <p className="text-xs text-muted">已完成</p>
        <p className="mt-1 text-2xl font-bold tabular-nums">{completedCount}</p>
      </div>
    </div>

    <div className="mt-5 flex items-center justify-between gap-3 text-sm">
      <span className="text-muted">平均学习进度</span>
      <span className="font-bold tabular-nums">{overallProgress}%</span>
    </div>
    <div className="mt-2">
      <ProgressBar value={overallProgress} label="平均学习进度" />
    </div>
  </section>

  <section className="rounded-2xl border border-line bg-surface p-5">
    <h2 className="text-base font-bold">今日小目标</h2>
    <p className="mt-2 text-sm leading-6 text-muted">
      复习一个布局概念，完成一个练习，并在手机和桌面视口各检查一次。
    </p>
    <div className="mt-4 rounded-xl border border-line p-3">
```

```
            <p className="text-sm font-semibold">动手检查清单</p>
            <ul className="mt-3 space-y-2 text-sm text-muted">
              <li className="flex items-start gap-2"><span className="text-brand" aria-hidden="true">✓</span><span>确认小屏没有横向溢出</span></li>
              <li className="flex items-start gap-2"><span className="text-brand" aria-hidden="true">✓</span><span>用键盘检查焦点状态</span></li>
              <li className="flex items-start gap-2"><span className="text-brand" aria-hidden="true">✓</span><span>切换浅色和深色主题</span></li>
            </ul>
          </div>
        </section>
      </aside>
    </main>

    <footer className="border-t border-line">
      <div className="mx-auto flex max-w-6xl flex-col gap-2 px-4 py-6 text-sm text-muted sm:flex-row sm:items-center sm:justify-between sm:px-6 lg:px-8">
        <span>学习空间 · Tailwind CSS 实践项目</span>
        <span>保持一致，从明确的设计规则开始。</span>
      </div>
    </footer>
  </div>
);
}
```

### 代码说明

- `statusClasses` 是完整的静态映射，构建时能找到每种状态对应的类名。
- `ProgressBar` 使用 aria progressbar 语义，并把百分比动态值放在 inline style 中，没有拼出动态 utility 类名。
- `group` 让卡片中的标题与箭头响应父卡片的悬停状态。
- `Button` 把共用交互状态封装为组件 API；页面决定按钮的位置和宽度。
- 外层 `max-w-6xl mx-auto px-*` 控制页面内容宽度和留白。
- 主区域在 `lg` 断点切换到内容区加侧栏，小屏维持单列。
- `bg-canvas`、`bg-surface`、`text-ink` 和 `text-muted` 使用语义 token。切换 `.dark` 后它们从 CSS 变量读取新的主题值。
- 深色模式按钮有清楚的标签和 `aria-pressed` 状态。生产应用还可以加上保存用户偏好的逻辑。

这里的“继续学习”链接和按钮只做演示导航，没有连接真实课程服务。不要把演示交互当成已经实现了真实业务逻辑。

## 12.6 运行与验收

运行：

```
npm run dev
```

检查下面这些真实场景，而不是只确认“页面看起来不错”：

1. **窄屏：** 将浏览器缩到约 360px。标题、按钮、卡片是否仍可读？是否出现整页横向滚动？
2. **中屏：** 观察 Grid 从一列变两列后的宽度与间距。
3. **宽屏：** 观察主内容与侧栏能否协调排列，侧栏没有被挤压。
4. **深色主题：** 切换主题，检查正文、次级文字、卡片、边框、徽标和进度条。
5. **鼠标与键盘：** hover 卡片；用 Tab 导航，确认焦点可见；尝试使用按钮。

6. **状态：** 检查已完成与学习中徽标。不要仅以颜色区分状态，示例中也有文本标签。
7. **构建：** 运行 `npm run build`，确认生产构建通过。

如果页面没有颜色，先检查 `index.css` 是否被引入、主题 utility 是否拼写正确。如果只有部分状态类没有效果，检查完整类名是否存在于源码。若页面在深色主题时仍出现白色面板，检查该面板是否还用了硬编码的 `bg-white` 而不是 `bg-surface`。

---

# 第 13 章：常见故障排查

## 13.1 Tailwind 类名写了，但样式没有生效

按顺序检查：

1. 类名是否真实存在且拼写正确？例如 `text-brand` 对应主题变量是否定义？
2. 样式表是否被应用入口导入？
3. Vite 配置有没有注册 `@tailwindcss/vite`？
4. 当前项目是 v4 还是 v3？你是否使用了错误版本的配置语法？
5. 类名是否由字符串拼接生成，导致扫描器看不到完整 token？
6. CSS 是否被其他规则覆盖，或者当前状态（如 hover）并没有发生？

不要一次性大改所有文件。每次检查一项，缩小问题范围。

## 13.2 页面出现横向滚动条

常见原因包括：固定宽度子元素、长单词或长 URL、Grid 轨道最小尺寸、未允许收缩的 Flex 子项、绝对定位元素超出容器。可以先在浏览器开发者工具查看超出视口的元素，再考虑 `min-w-0`、`break-words`、合理的 max-width、局部 overflow 或响应式切换。

## 13.3 深色模式只有一部分改变

优先检查是否混用了两套做法：有的组件使用语义 token，有的组件仍直接写 `bg-white`、`text-slate-900`。如果采用语义 token，普通组件应尽可能从 token 取色；只有特殊差异才用 `dark:`。此外，确认 `.dark` 类真的加到了 `html` 或符合 `@custom-variant` 的祖先元素上。

## 13.4 `hover:` 正常，但键盘看不到焦点

检查是否误用了 `outline-none`，或者只有 `hover:` 没有 `focus-visible:`。按 Tab 键测试，而不是只用鼠标。对比度不足也会让焦点看似不存在。

## 13.5 自定义组件的 `className` 覆盖不了默认样式

不能依赖 className 字符串的排列顺序决定最终 CSS。如果组件 API 真正允许调用处改变某个默认 utility，可以使用 `tailwind-merge` 处理它支持的冲突类别；更重要的是明确哪些样式允许覆盖、哪些属于组件不可变的行为约束。

## 13.6 从 v3 复制教程后构建出错

检查 `package.json` 的 Tailwind 主版本。v3 的 `@tailwind` 指令、`content` 和某些配置项，不应直接被当作 v4 的默认做法。不要为了让一段旧教程工作就同时安装多套互相冲突的集成插件。按锁定主版本重新核对官方安装流程。

### 13.7 什么时候不该继续增加 utility

如果同一个元素需要几十个与业务状态相关的条件，或者同一段样式已经在多个页面重复出现，那么问题可能不是缺少某个 Tailwind 类，而是需要重新组织组件、主题 token、状态模型或布局职责。Utility-first 仍然需要良好的组件设计。

---

# 第 14 章：学习完成后的工程化自检清单

完成本教程后，请逐项检查你的项目。不是为了让类名越多越好，而是确认每个样式选择都有可解释的理由。

## 样式基础

- ☐ 我能说出 `flex`、`grid`、`gap-*`、`p-*`、`mx-auto`、`max-w-*` 分别解决什么问题。
- ☐ 我能区分 padding、margin、gap，以及 `justify-*` 与 `items-*`。
- ☐ 我知道如何用 `min-w-0`、`shrink-0`、`truncate` 或 `overflow-x-auto` 处理不同的溢出情形。
- ☐ 我只在有明确需求时使用任意值，而不是把所有尺寸都写成方括号数值。

## 响应式

- ☐ 我从移动端默认样式开始，再添加 `sm:`、`md:`、`lg:` 等必要变化。
- ☐ 我测试了真实窄屏，而不是只看桌面浏览器。
- ☐ 我知道何时应该改变 Grid 列数、Flex 方向、宽度或可见性。
- ☐ 我能分辨视口断点和容器查询。

## 状态与可访问性

- ☐ 可交互元素有 hover、focus-visible、active 等合理反馈。
- ☐ 禁用状态同时有真实 `disabled` 或适当的交互语义，不只是降低透明度。
- ☐ 我理解 `group-hover` 和 `peer-invalid` 的关系与 DOM 限制。
- ☐ 用键盘操作时，焦点清楚可见。
- ☐ HTML 元素语义正确，表单输入有关联标签，图片 alt 合适。

## 主题

- ☐ 新项目的主题令牌放在 v4 的 CSS-first 配置中。
- ☐ 我使用语义颜色 token 表达 `canvas`、`surface`、`ink`、`muted`、`line`、`brand` 等用途。
- ☐ 深色主题中不只背景变化，文字、边框、徽标与交互状态也经过检查。
- ☐ 我避免把一堆硬编码颜色散落在重复组件里。

## 组件与动态类名

- ☐ 重复使用的组件有清晰的 props，例如 `variant`、`disabled` 和 `className`。
- ☐ 条件 className 使用完整字符串的映射，不拼接 `bg-${color}-600`。
- ☐ 动态数字（例如进度百分比）用 style 或 CSS 变量表达，而不是伪造动态 utility 名称。
- ☐ 必要时我了解 v4 的 `@source` 与 `@source inline()`。
- ☐ 我没有把 JS 运行时业务逻辑藏进一个巨大的 className 条件表达式里。

### 版本和构建

- ☐ 我确认了项目 Tailwind 主版本，并使用该版本的官方安装文档。
- ☐ 我理解 v4 的 `@import "tailwindcss"` 与 v3 的 `@tailwind` 指令存在差异。
- ☐ 我检查了锁文件，并确认开发与生产构建均能通过。
- ☐ 我了解 v4 对浏览器版本的要求，确认其符合项目支持范围。

---

# 第 15 章：下一步如何继续学习

完成仪表盘后，不建议立即去背几百个类名。更有效的进阶顺序是：

1. **用真实页面练习布局。** 分别重建一个登录页、表单页、产品列表页和管理后台。每个页面都从小屏布局开始，逐步加宽。
2. **定义自己的设计令牌。** 不要只使用默认的颜色名称。根据项目需求定义语义颜色、字号层级、间距、圆角、阴影和断点。
3. **建立组件库边界。** 抽象 Button、Input、Badge、Card、Dialog 等真正重复的视觉与交互模式；不要把每个容器都变成组件。
4. **学习状态管理与可访问性。** 学会表单校验、弹窗焦点管理、标签页语义、加载和错误状态。样式只表现状态，不负责创造业务状态。
5. **学习普通 CSS 的复杂能力。** 深入理解选择器、级联层、Grid、Flex、容器查询、变量和媒体查询。当 utility 无法清楚表达意图时，写正确的 CSS 比硬套类名更好。
6. **每次升级核对官方文档。** Tailwind 功能、配置和浏览器支持会随版本迭代；旧文章即使仍可搜索到，也可能讲的是另一主版本。

Tailwind CSS 学习的终点不是记住多少类名，而是拥有一种稳定的工作方式：以清晰的语义结构组织 HTML，以 utility 快速表达布局与视觉，以 design token 管理主题，以组件 API 管理重复模式，以可访问的状态与真实交互保证界面可靠，并在项目锁定的主版本下持续维护。

---

# 官方资料索引

以下链接对应本教程使用的官方文档。遇到版本差异时，优先打开对应章节重新核对。本文以官方文档在 2026-10-22 展示的 Tailwind CSS v4.3 文档为主。

1. Getting Started / Installation: https://tailwindcss.com/docs/installation
2. Install with Vite: https://tailwindcss.com/docs/installation/using-vite
3. Utility classes: https://tailwindcss.com/docs/styling-with-utility-classes
4. Responsive design: https://tailwindcss.com/docs/responsive-design
5. Hover, focus, and other states: https://tailwindcss.com/docs/hover-focus-and-other-states
6. Dark mode: https://tailwindcss.com/docs/dark-mode
7. Theme variables: https://tailwindcss.com/docs/theme
8. Colors: https://tailwindcss.com/docs/colors
9. Detecting classes in source files: https://tailwindcss.com/docs/detecting-classes-in-source-files
10. Adding custom styles: https://tailwindcss.com/docs/adding-custom-styles
11. Upgrade guide: https://tailwindcss.com/docs/upgrade-guide
12. Compatibility: https://tailwindcss.com/docs/compatibility
13. Tailwind CSS v3 configuration: https://v3.tailwindcss.com/docs/configuration
14. Tailwind CSS v3 content configuration: https://v3.tailwindcss.com/docs/content-configuration